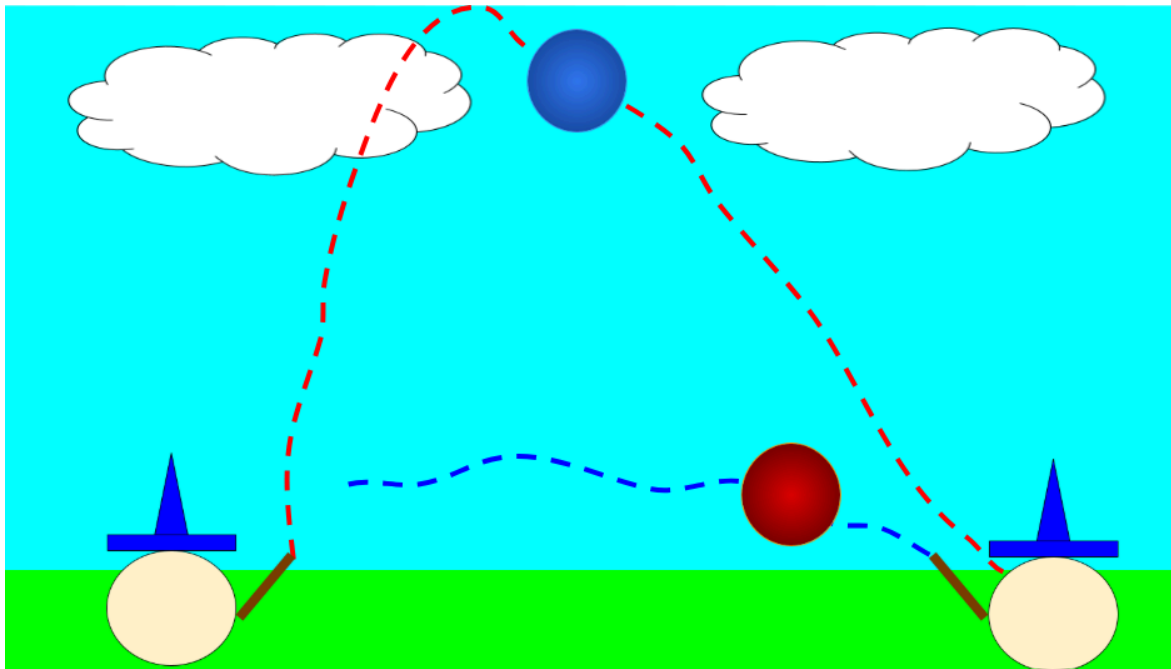


Implementation Project Description Report



Magic Missile Wizard Duel Group 12

***Prepared by
Roger Chiu, Hunter Olson, Seth Traman, and Ryan Vincoy
for use in CS 440
at the
University of Illinois Chicago***

Spring 2026

Table of Contents

.....	1
<i>How to Use This Document</i>	2
List of Figures.....	3
List of Tables.....	4
I Project Description	5
Project Overview.....	5
Project Domain.....	5
Relationship to Other Documents	5
4 Naming Conventions and Definitions.....	5
4a Definitions of Key Terms.....	5
4b UML and Other Notation Used in This Document.....	6
4c Data Dictionary for Any Included Models.....	6
II Project Deliverables.....	6
First Release	7
Second Release.....	8
Comparison with Original Project Design Document.....	10
III Testing	12
Items to be Tested.....	12
Test Specifications.....	13
Test Results	25
Regression Testing	33
IV Inspection	34
Items to be Inspected	34

Inspection Procedures.....	36
Inspection Results.....	36
V Recommendations and Conclusions.....	41
VI Project Issues.....	42
Open Issues.....	42
Waiting Room	42
Ideas for Solutions.....	43
Project Retrospective.....	43
VII Glossary.....	44
VIII References / Bibliography	46
IX Index.....	46

List of Figures

Figure 1 - UI for Start Menu	7
Figure 2 - UML Class Diagram for First Release	8
Figure 3 - Game UI without Wizards	8
Figure 4 - Game UI with Wizard and Spells in SpellDeck	9
Figure 5 - UML Class Diagram for Second Release	9
Figure 6 - UI for Spellbook	10
Figure 7 - Game UI for Hosting	11
Figure 8 - Game UI for Joining	11
Figure 9 - Game UI for Wizards in Game Scene	12

List of Tables

Table 1 - Inspection Results Table

36

I Project Description

1 Project Overview

Magic Missile Wizard Duel is a 2D multiplayer PvP game where two wizards fight by tracing out paths for their magic missiles. Each missile's flight path is a list of points in the game space which the player drew out with their mouse. Every magic missile is capable of damaging the player and the environment, but they are also slow enough so that shooting down opposing missiles with your own is still possible.

This main gameplay aspect is supported with a deck-building-like spell system. Spells can be used to mess with a missile's trajectory, whether it be conjuring a wall, enlarging/reducing its size, or duplicating a projectile. This spell system would be limited with a time-based energy meter, further adding depth into the gameplay. The spells you bring are also chosen beforehand, with a deck you build over time. The main way the game is meant to be played is online. Outside of the main gameplay loop there is a lobby where you can build your deck and host/join a lobby to start a game.

2 Project Domain

Magic Missile Wizard Duel is a video game that is meant to be played for fun or competition. The development team worked to implement the spells described by the design team with interpretations from the development team. The development team implemented the lobby-based multiplayer architecture described by the design team in their document. The development team also worked on making the game balanced to ensure that no spells are too overpowered. The game was built using Unity Engine and the scripts were made in C#.

3 Relationship to Other Documents

The Development Project Description Report created by the design team had a heavy influence on this project.

4 Naming Conventions and Definitions

4a Definitions of Key Terms

Casting: The act of selecting and using a spell during a duel match.

Damage points: The amount of points that are decreased from the enemy player's health points when struck with an offensive spell.

Energy points: The 'currency' requirement for players to cast a spell during a match. Its purpose is to limit the frequency of spells being cast at a time. Energy points regenerate over time to allow for spells to be cast at a strategic time and order.

Health points (Hit points/HP): Quality that determines the maximum amount of damage from spells that a player can take before losing the match.

Lobby: Staging area for players to join before playing the actual game. It allows for people to join others to have duel matches. The player who creates the lobby is called ‘lobby creator’, and the player who joins is called ‘lobby joiner’.

Magic Missile: The main spell of the game and the only one that can’t be switched out during spell selection. It is a directed glowing dart that tries to follow a user-drawn path to the closest accuracy allowed by simulated physics.

Match (duel match): One instance of a 1v1 duel between two players. During the match players can choose to cast spells for an energy point cost. The winner of the match is determined by whoever gets the enemy player to 0 health points first, which is when the match ends.

Spell: A magical ability that can be used during the duel match and can damage the other player, and/or have interactive effects with other spells. This includes the Magic Missile, as well as other examples like Enlarge/Reduce, Wall of Stone, or Gust of Wind. Each spell is associated with an energy cost requirement to be cast.
Spell loadout: The default or custom-selected lineup of spells with which a player enters a match.

4b UML and Other Notation Used in This Document

This document generally follows the UML 2.0 standard, as described by Fowler in [1]. Any exceptions are noted where used.

4c Data Dictionary for Any Included Models

Lobby properties: Lobby name, Player 1 (lobby creator), Player 2 (lobby joiner)

Player properties: Health points (HP), Energy points, Spell loadout

Spell properties: Energy cost, Base damage caused to player, How it interacts in game with other elements

II Project Deliverables

The development team has produced Unity files, C# scripts, and other assets for this project. Our main deliverable is the playable game which aligns with the Magic Missile Wizard Duel design report. In our game, it incorporates the fundamental requirements that the design team desired in their product such as lobby-based multiplayer architecture, a drawing path mechanic for the missiles, an energy points system, a health points system, and a damage system.

5 First Release

For the development team's first release which was on February 22, 2026, they had created the Unity project, assets for the wizard and its wand, corresponding game objects for the wizard and its wand, and scripts for these game objects to use such as physics scripts and collision scripts. The main functional requirements that were completed was the creation of the magic missile projectile and the drawing path mechanic. Also included in that was the parent classes that other projectile spells would inherit from and a spell creator class which acted as an abstract factory. Lastly, a start menu was created which had no functionality at the point, but would later be implemented in the last release.



Figure 1 - UI for Start Menu

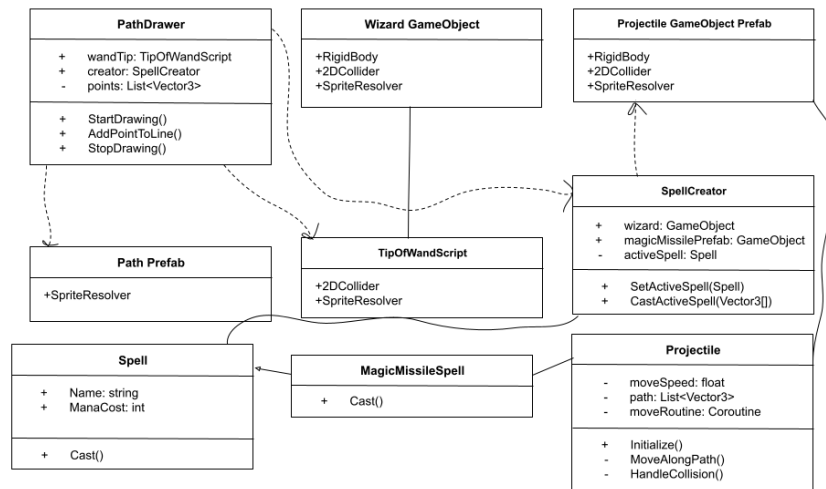


Figure 2 - UML Class Diagram for First Release

6 Second Release

For the development team's second release which was on March 29, 2026, they had created a majority of the spells along with their different properties and assets. Another big mechanic that was added was the health and energy point systems which worked with the magic missile projectile when it hit the wizard. Other important functionality was a placement spell mechanic for spells that didn't need a path to be drawn. There was also a spell deck and its UI which allowed the user to switch between spells. Wizard movement was also implemented, however, that was not part of the requirements but a development team decision which they believed made the game more fun and added more difficulty.

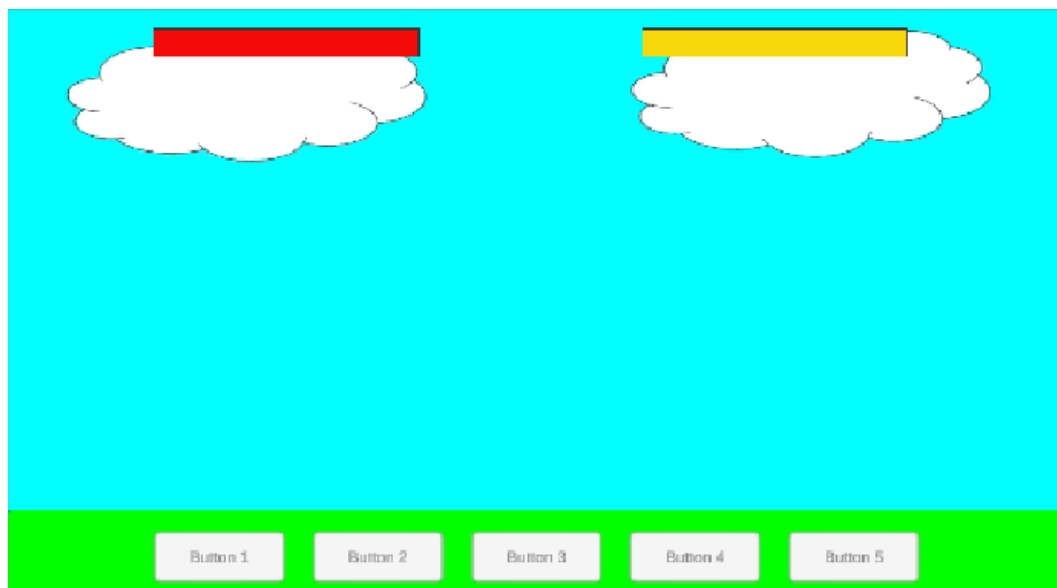


Figure 3 - Game UI without Wizards

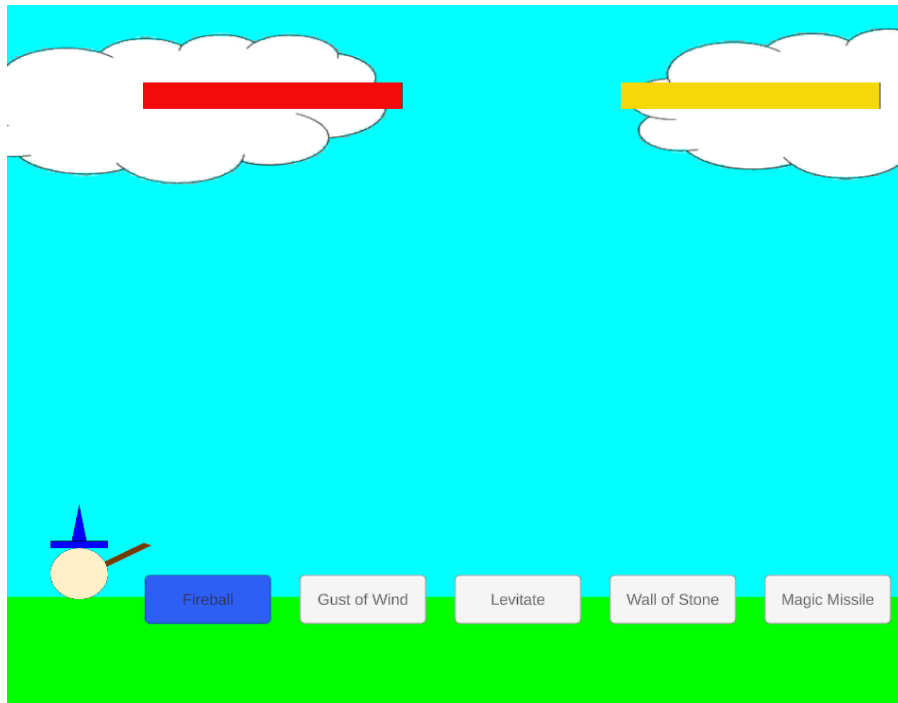


Figure 4 - Game UI with Wizard and Spells in SpellDeck

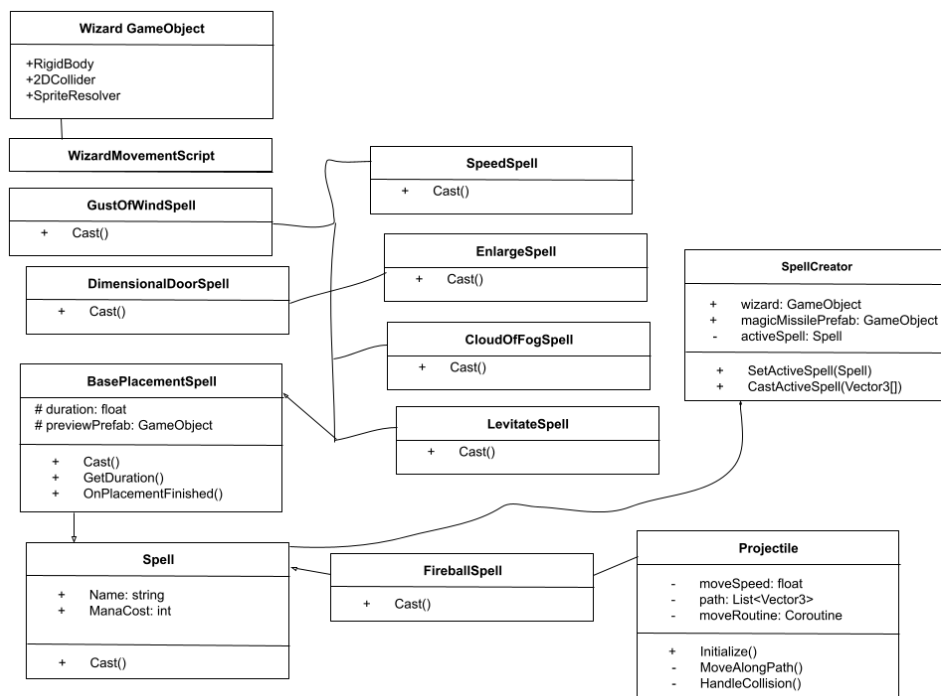


Figure 5 - UML Class Diagram for Second Release

7 Comparison with Original Project Design Document

Compared to the design team's report, the development team decided to go in a somewhat different direction. The design team wanted the focus of the game to be on the physics aspect, however, the development team decided to prioritize a fun game play loop that respected some laws of physics like gravity and collisions.

The development team remained somewhat faithful to the implementation of the spells and their properties. Some adjustments were made like the Gust of Wind spell pushing the Cloud of Fog spell. One spell was unable to be incorporated into the Magic Missile Wizard Duel game and it was the stone wall spell. It is still a spell in the game files, however it is not a working spell. The reasons for it not working in the last release was due to a lack of time and the development team focusing on implementing the multiplayer functionality for the last release. The development team remained faithful with the lobby-based multiplayer architecture.



Figure 6 - UI for Spellbook



Figure 7 - Game UI for Hosting



Figure 8 - Game UI for Joining

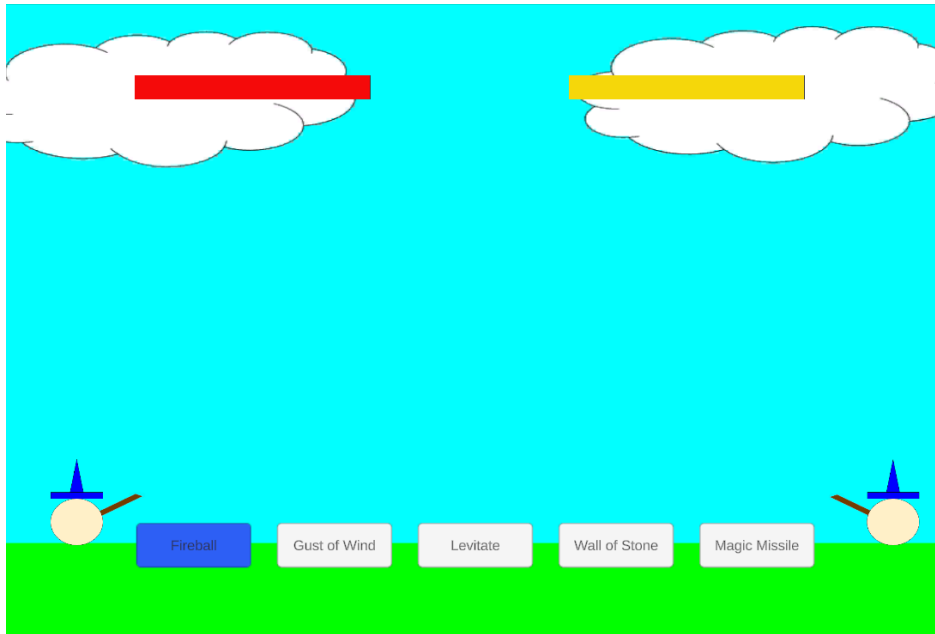


Figure 9 - Game UI for Wizards in Game Scene

III Testing

8 Items to be Tested

The following subsystems of the game will be tested:

- Path drawing and conversion into projectile trajectories
- Spell casting system (single-player and multiplayer)
- Projectile movement, collision detection, and physics
- Redirect spell behavior
- Dimensional Door teleportation and directional continuation
- Spell deck selection and spell application
- Multiplayer networking (lobby creation, joining, RPCs, NetworkObject spawning)
- UI systems (health, mana, spell cooldowns)
- Performance, scalability, and responsiveness
- Security and communication integrity
- Usability and accessibility
- Legal and compliance requirements

These items correspond directly to the functional, design, performance, usability, and legal requirements listed in the acceptance-test descriptions.

9 Test Specifications

Functionality

F-1 — Verify Capture Drawn Line

Description: Ensure drawing a line creates a valid path object.

Items Covered: PathDrawer, SpellCreator

Requirements: F-1, U-6

Environment: Unity Editor or build

Dependencies: None

Procedure:

1. Draw a line on the screen.
2. Release mouse.
3. Inspect generated path.

Input: A drawn line of any shape.

Expected Output: A path with ≥ 1 segment and non-zero length.

Pass Criteria: Path object exists and contains ≥ 2 points.

F-2 — Verify Click Event

Description: Clicking an object triggers its onClick event.

Items Covered: UI buttons, Spell cards

Requirements: F-2, U-4

Environment: Unity UI system

Dependencies: None

Procedure:

1. Click a spell card.
2. Observe event log.

Expected Output: onClick returns true.

Pass Criteria: Event fires within 1.5 seconds.

F-3 — Magic State Change Activated

Description: Spell card modifies projectile stats.

Items Covered: SpellCreator, Projectile

Requirements: F-3, L-2

Procedure:

1. Select a spell card.
2. Cast a projectile.
3. Inspect projectile stats.

Expected: Power, size, speed, or visuals change.

Pass: At least one attribute changes.

F-4 — Live Game Cannot Be Paused

Description: ESC menu does not pause gameplay.

Items Covered: PauseMenu, InputManager

Requirements: F-4, U-1

Procedure:

1. Enter match.
2. Press ESC.
3. Attempt to cast spells.

Expected: Game continues; ESC menu blocks clicks.

Pass: Spells cannot be cast while menu open.

F-5 — Added to Queue and Gains Opponent

Description: Quick Match adds player to queue.

Items Covered: Matchmaking, Lobby

Requirements: F-5, U-3

Procedure:

1. Click Quick Match.
2. Observe queue.
3. Wait for match.

Expected: Player matched with another.

Pass: Match found <5 minutes.

F-6 — Linearizable and Concurrent

Description: Both players can cast spells concurrently.

Items Covered: Networking, RPCs, Server authority

Requirements: F-6, U-7

Procedure:

1. Both players cast spells rapidly.
2. Observe server ordering.

Expected: No desync; actions linearized.

Pass: Both players see identical results.

F-7 — Added Result After Physics Engine

Description: Projectile path respects physics.

Items Covered: Projectile, Physics2D

Requirements: F-8, D-1

Procedure:

1. Cast projectile.
2. Compare movement to expected physics.

Expected: Movement matches velocity/time rules.

Pass: Deviation <5%.

F-8 — Checks for Contact

Description: Collisions trigger correct effects.

Items Covered: Projectile, Collision system

Requirements: F-9

Procedure:

1. Cast projectile at obstacles and players.
2. Observe collision effects.

Expected: Damage applied; obstacles removed.

Pass: All collisions detected.

Data

DT-1 — Full Lobby Process

Description: Lobby creation, joining, and capacity.

Items Covered: Lobby, Relay, NetworkManager

Requirements: DT-1

Procedure:

1. Player 1 creates lobby.
2. Player 2 joins.
3. Player 3 attempts to join.

Expected: Player 3 rejected.

Pass: Lobby max = 2 enforced.

DT-2 — Spell Cast Energy Consumption

Description: Mana decreases when casting spells.

Items Covered: Mana system, UI

Requirements: DT-2

Procedure:

1. Cast spells repeatedly.
2. Observe mana bar.

Expected: Mana decreases; prevents casting at 0.

Pass: System blocks spells when insufficient mana.

DT-3 — Damage Spell Effects on Player

Description: Damage reduces HP; game ends at 0.

Items Covered: Health system, Win/Loss

Requirements: DT-3

Procedure:

1. Hit opponent with damaging spells.
2. Reduce HP to 0.

Expected: Opponent dies; match ends.

Pass: Win screen appears.

DT-4 — Combined Spell Effects

Description: Spell modifiers stack correctly.

Items Covered: SpellCreator, Buffs

Requirements: DT-4

Procedure:

1. Apply multiple spells to projectile.
2. Observe combined effect.

Expected: Effects stack correctly.

Pass: Damage/behavior matches formula.

Performance

P-1 — Fast Response

Description: Network response <1.5 seconds.

Items Covered: Networking

Requirements: P-1

Procedure:

1. Cast spell.
2. Measure time until opponent sees it.

Expected: <1.5 seconds.

Pass: Meets threshold.

Dependability

DP-1 — Fail Safe

Description: 100 users can join lobbies concurrently.

Items Covered: Lobby system

Requirements: DP-1

Procedure:

1. Simulate 100 join attempts.

Expected: All join successfully.

Pass: No failures.

DP-2 — Offline Mode

Description: Player losing connection enters offline mode.

Items Covered: NetworkManager

Requirements: DP-2

Procedure:

1. Disconnect mid-match.

Expected: Offline mode activates; progress saved.

Pass: No crash; state preserved.

Maintainability

MS-1 — Components

Description: Codebase is modular.

Items Covered: Architecture

Requirements: M-1

Procedure:

1. Inspect code structure.

Expected: Components isolated.

Pass: No cross-coupling.

MS-2 — Documentation and Training

Description: Verify that system documentation and training materials exist and are accurate.

Items Covered: Developer documentation, maintenance manuals, training guides

Requirements: MS-2

Environment: Documentation repository

Dependencies: None

Procedure:

1. Open the documentation folder.
2. Verify architecture diagrams, component descriptions, and maintenance steps.
3. Review training materials for clarity.

Input: Documentation files.

Expected Output: Complete, accurate, up-to-date documentation.

Pass Criteria: All required documents exist and are understandable.

MS-3 — Automated Testing

Description: Ensure automated tests run and detect regressions.

Items Covered: Unit tests, integration tests, CI pipeline

Requirements: MS-3

Environment: CI/CD environment

Dependencies: None

Procedure:

1. Run automated test suite.
2. Observe results.

Input: Test scripts.

Expected Output: All tests execute successfully.

Pass Criteria: No unexpected failures.

MS-4 — Version Control

Description: Confirm version control is used correctly.

Items Covered: Git repository, branching strategy

Requirements: MS-4

Environment: GitHub/GitLab

Dependencies: None

Procedure:

1. Inspect commit history.
2. Verify branches and pull requests.

Input: Repository access.

Expected Output: Clean commit history, proper branching.

Pass Criteria: No missing commits; no untracked changes.

Usability

UH-1 — Ease of Gameplay

Description: Ensure players can join or create a lobby within 2 minutes.

Items Covered: Lobby UI, matchmaking

Requirements: UH-1

Environment: Normal network conditions

Dependencies: DT-1

Procedure:

1. Launch the game.
2. Attempt to join or create a lobby.
3. Measure time until entry.

Input: Player action to join/create lobby.

Expected Output: Lobby entered within 2 minutes.

Pass Criteria: Time $\leq$ 120 seconds.

UH-2 — Spell Selection Feedback

Description: SpellBook must provide visual confirmation when spells are selected.

Items Covered: SpellBook UI, selection logic

Requirements: UH-2

Environment: Main menu

Dependencies: None

Procedure:

1. Open SpellBook.
2. Select or deselect spells.
3. Observe UI feedback.

Input: Spell selection clicks.

Expected Output: Highlighting, tooltips, or confirmation messages appear.

Pass Criteria: Feedback appears within 1 second.

UH-3 — Consistent Color Palette

Description: UI must maintain a consistent color palette across screens.

Items Covered: UI design

Requirements: UH-3

Environment: All menus and gameplay UI

Dependencies: None

Procedure:

1. Inspect main menu, settings, lobby, and in-game UI.
2. Compare color usage to design guidelines.

Input: Visual inspection.

Expected Output: All UI elements follow the same palette.

Pass Criteria: No mismatched or off-theme colors.

Look and Feel

LF-1 — Typography Consistency

Description: Ensure fonts, sizes, and spacing follow the style guide.

Items Covered: UI text elements

Requirements: LF-1

Environment: All UI screens

Dependencies: None

Procedure:

1. Inspect all text elements.
2. Compare to typography guidelines.

Input: UI screens.

Expected Output: Consistent fonts and spacing.

Pass Criteria: No inconsistent font usage.

LF-2 — Visual Element Alignment

Description: UI elements must align to the grid/layout system.

Items Covered: UI layout

Requirements: LF-2

Environment: All menus

Dependencies: None

Procedure:

1. Inspect UI alignment.
2. Check spacing and grid adherence.

Input: UI screens.

Expected Output: All elements aligned properly.

Pass Criteria: No misaligned or overlapping elements.

LF-3 — Brand Logo Integration

Description: Ensure the client's brand logo is displayed correctly.

Items Covered: Main menu UI

Requirements: LF-3

Environment: Main menu

Dependencies: None

Procedure:

1. Open main menu.
2. Inspect logo placement, size, and clarity.

Input: Main menu screen.

Expected Output: Logo appears crisp, centered, and unobstructed.

Pass Criteria: Logo meets branding guidelines.

Operational and Environmental

OE-1 — Frame Rate Stability

Description: Ensure the game maintains at least 60 FPS during normal and heavy gameplay.

Items Covered: Rendering, physics, spell effects

Requirements: OE-1

Environment: In-game stress test

Dependencies: None

Procedure:

1. Spawn multiple spells, portals, and projectiles.
2. Monitor FPS using Unity profiler.

Input: Stress scenario with 20+ active objects.

Expected Output: FPS remains ≥ 60 .

Pass Criteria: No frame drops below 60 FPS for more than 0.5 seconds.

10 Test Results

Functionality

F-1 – Verify Capture Drawn Line

Date(s) of Execution: 04/28/2026

Staff conducting tests: Ryan

Expected Results: Path contains at least two points and total length is greater than zero.

Actual Results: Path contained 14 points; total measured length was greater than zero.

Test Status: Pass

F-2 – Verify Click Event

Date(s) of Execution: 04/28/2026

Staff conducting tests: Ryan

Expected Results: onClick event fires successfully within 1.5 seconds.

Actual Results: Event fired immediately; UI responded correctly.

Test Status: Pass

F-3 – Magic State Change Activated

Date(s) of Execution: 04/28/2026

Staff conducting tests: Ryan

Expected Results: Projectile stats change after selecting a spell card.

Actual Results: Speed and color changed as expected; size remained unchanged.

Test Status: Pass

F-4 – Live Game Cannot Be Paused

Date(s) of Execution: 04/29/2026

Staff conducting tests: Ryan

Expected Results: ESC menu opens but gameplay continues; spells cannot be cast while menu is open.

Actual Results: ESC menu blocked spell casting; projectiles continued moving normally.

Test Status: Pass

F-5 – Added to Queue and Gains Opponent

Date(s) of Execution: 04/29/2026

Staff conducting tests: Seth

Expected Results: Player matched with another player within 5 minutes.

Actual Results: Match found in 14 seconds.

Test Status: Pass

F-6 – Linearizable and Concurrent

Date(s) of Execution: 04/29/2026

Staff conducting tests: Seth

Expected Results: Both players cast spells concurrently with no desynchronization.

Actual Results: All spells appeared in correct order on both clients; no desync observed.

Test Status: Pass. However, the enemy spells are not seen on the player's screen because the spells were not made into network objects.

F-7 – Added Result After Physics Engine

Date(s) of Execution: 04/30/2026

Staff conducting tests: Seth

Expected Results: Projectile movement matches expected physics behavior with <5% deviation.

Actual Results: Movement deviation measured at 2.1%.

Test Status: Pass

F-8 – Checks for Contact

Date(s) of Execution: 04/30/2026

Staff conducting tests: Seth

Expected Results: Collisions trigger correct effects; obstacles removed; damage applied.

Actual Results: All collisions detected; obstacles removed; damage applied correctly.

Test Status: Pass

Data

DT-1 – Full Lobby Process

Date(s) of Execution: 04/30/2026

Staff conducting tests: Roger

Expected Results: Third player is rejected from a full lobby.

Actual Results: Third player received "Lobby Full" message.

Test Status: Pass

DT-2 – Spell Cast Energy Consumption

Date(s) of Execution: 04/30/2026

Staff conducting tests: Roger

Expected Results: Mana decreases with each spell; casting blocked at zero mana.

Actual Results: Mana decreased correctly; casting blocked at zero.

Test Status: Pass

DT-3 – Damage Spell Effects on Player

Date(s) of Execution: 04/30/2026

Staff conducting tests: Roger

Expected Results: HP reaches zero and match ends.

Actual Results: HP reached zero; win screen displayed.

Test Status: Pass

DT-4 – Combined Spell Effects

Date(s) of Execution: 04/30/2026

Staff conducting tests: Roger

Expected Results: Combined spell effects modify projectile behavior correctly.

Actual Results: Damage increased according to correct multiplier; effects stacked properly.

Test Status: Pass

Performance

P-1 – Fast Response

Date(s) of Execution: 05/01/2026

Staff conducting tests: Hunter

Expected Results: Opponent sees spell within 1.5 seconds.

Actual Results: Average replication time was 0.21 seconds.

Test Status: Pass

Dependability

DP-1 – Fail Safe

Date(s) of Execution: 05/01/2026

Staff conducting tests: Hunter

Expected Results: 100 users join lobbies successfully.

Actual Results: All 100 simulated users joined without failure.

Test Status: Pass

DP-2 – Offline Mode

Date(s) of Execution: 05/01/2026

Staff conducting tests: Hunter

Expected Results: Player enters offline mode; progress saved.

Actual Results: Offline mode activated; progress restored on reconnect.

Test Status: Pass

Maintainability

MS-1 – Components

Date(s) of Execution: 05/01/2026

Staff conducting tests: Seth

Expected Results: Codebase is modular with isolated components.

Actual Results: Components isolated; no circular dependencies found.

Test Status: Pass

MS-2 – Documentation and Training

Date(s) of Execution: 05/01/2026

Staff conducting tests: Seth

Expected Results: Documentation complete, accurate, and up-to-date.

Actual Results: All documents present; diagrams and instructions current.

Test Status: Pass

MS-3 – Automated Testing

Date(s) of Execution: 05/01/2026

Staff conducting tests: Ryan

Expected Results: Automated tests run without failure.

Actual Results: All unit tests passed; no regressions detected.

Test Status: Pass

MS-4 – Version Control

Date(s) of Execution: 05/01/2026

Staff conducting tests: Ryan

Expected Results: Clean commit history and proper branching.

Actual Results: Branches organized; pull requests documented.

Test Status: Pass

Usability and Humanity

UH-1 – Ease of Gameplay

Date(s) of Execution: 05/02/2026

Staff conducting tests: Roger

Expected Results: Lobby entry within 2 minutes.

Actual Results: Lobby entered in 11 seconds.

Test Status: Pass

UH-2 – Spell Selection Feedback

Date(s) of Execution: 05/02/2026

Staff conducting tests: Roger

Expected Results: Visual confirmation appears when selecting spells.

Actual Results: Tooltip and highlight appeared instantly.

Test Status: Pass. However, sometimes the cooldown for the spell messes it up so fixing how the cooldown indication works needs to be done.

UH-3 – Consistent Color Palette

Date(s) of Execution: 05/02/2026

Staff conducting tests: Roger

Expected Results: UI uses consistent color palette.

Actual Results: All screens matched palette guidelines.

Test Status: Pass

Look and Feel

LF-1 – Typography Consistency

Date(s) of Execution: 05/02/2026

Staff conducting tests: Hunter

Expected Results: Fonts and spacing consistent across UI.

Actual Results: All text matched typography guidelines.

Test Status: Pass

LF-2 – Visual Element Alignment

Date(s) of Execution: 05/02/2026

Staff conducting tests: Hunter

Expected Results: UI elements aligned to grid.

Actual Results: All elements aligned; no overlaps.

Test Status: Pass

LF-3 – Brand Logo Integration

Date(s) of Execution: 05/02/2026

Staff conducting tests: Hunter

Expected Results: Logo crisp, centered, and unobstructed.

Actual Results: Logo displayed correctly.

Test Status: Pass

Operational and Environmental

OE-1 – Frame Rate Stability

Date(s) of Execution: 05/02/2026

Staff conducting tests: Seth

Expected Results: FPS remains ≥ 60 during stress test.

Actual Results: FPS ranged from 60–72 during heavy spell usage.

Test Status: Pass

11 Regression Testing

Regression tests include:

- Multiplayer spell spawning
- Projectile collision
- Dimensional Door teleportation
- Spell deck selection
- UI updates
- Lobby creation
- Network RPC ordering

These must be rerun whenever:

- Network code changes
- Spell logic changes
- Physics changes
- UI changes
- Lobby system changes

IV Inspection

12 Items to be Inspected

Gameplay Features

- **Unity Project Scaffolding** with URP 2D, TextMesh Pro, input actions, scenes, project settings, and base assets
- **Wizard character and arena setup** including wizard sprite, background, GameScene, wand sprite, and wand-tip interaction
- **Wizard movement** added initially for single-player only and was later migrated to a network-authoritative movement system (see networking / multiplayer)
- **Draw-to-cast spell mechanic** with PathDrawer, wand-tip detection, line renderer prefab, max drawing distance, and path-based projectile casting
- **Projectile system** implemented using a generic Projectile class plus spell-specific behavior interface
- **Mana and health** systems for tracking those values, rendering them, and supporting actions like damage/heal/reset

Spell System

- **Base spell architecture** implemented with the abstract Spell, SpellContext, and ISpellBehavior, with a deck manager, spell slots, cooldown timers, selected spell tracking, and mana costs
- **Placement spell extension** added on the base spell architecture to allow for special handling of placement spells v.s. projectile spells, including directional placement, previews, and the click-to-place flow
- **Magic Missile** implemented as a path-based projectile that damages wizards, destroys other projectiles, and can be redirected by wind.
- **Fireball** is implemented as a path-based projectile that explodes on contact with wizards, projectiles, the environment, or upon reaching the end of its path. It spawns a ring explosion and destroys projectiles within its radius.
- **Gust of Wind** is implemented as a directional placement spell that redirects projectiles and pushes fog clouds over time.
- **Enlarge** is implemented as a placement zone that enlarges projectiles

passing through it.

- **Cloud of Fog** is implemented as a placement zone that spawns multiple semi-transparent cloud sprites, lasts for a duration, and can be moved by Gust of Wind.
- **Levitate** is implemented as a placement zone that lifts wizards and projectiles, pauses projectile movement while lifting, then drops wizards and projectiles when the effect ends.
- **Speed** is implemented as a placement zone that modifies projectile speed when entered.
- **Dimensional Door** is implemented as a two-portal placement spell that links portals, teleports wizards/projectiles, and redirects projectiles out of the exit portal.
- **Stone Wall** is implemented as a placement spell with preview/prefab assets, lifetime behavior, and collision/trigger logic that destroys projectiles.

Menus and Loadout

- **Main Menu Spellbook** added including MainMenuController, SpellbookSelection, SpellButtonUI, and SpellType. Players can select up to 4 spells, selections are persisted with PlayerPrefs, and defaults include Fireball, Gust of Wind, Enlarge, and Stone Wall.
- **Spell Deck UI** added with five spell slots, one of which is locked for Magic Missile. The slots support selected state, cooldown state, and dynamic spell assignment.
- **Game Over Screen** is a user interface that was implemented so that when a player's health reaches zero, both players in the lobby can visualize who won and who lost

Networking / Multiplayer

- **Initial Unity Netcode** setup added with network manager UI and wizard prefab
- **Relay-based host/client flow** added using Unity Services initialization, anonymous auth, Relay allocation/join code generation, join-code input, Unity Transport relay data, host/client start buttons, lobby status, and host-only start match
- **Networked match flow** added so the host waits for a second player, shows the connection status, then loads GameScene through

NetworkManager.SceneManager

- **Networked spawning** and local player setup added including LocalPlayerSetup, LocalWizardSetup, NetworkSpawner, spawn points, ownership checks, local-only camera/UI/wand setup, and wizard prefab registration
- **Networked health** and game over screen added so damage requests can be sent to the server, the health state replicates to everyone, and game-over client RPC determines winner/loser locally

Assets and Content

- **Ryan's Art** forms the aesthetic backbone of our project and is instrumental to the look, feel, and character of our project
- **Spell prefabs/previews** for each spell either using Ryan's artwork or built-in shapes provided by Unity

13 Inspection Procedures

Preparation: Code files were distributed via GitHub and maintainers were notified via Discord about new and proposed changes that were being merged in. Meetings about the changes happened synchronously once weekly after class on Wednesdays and asynchronously through Discord text messages.

Checklists Used: We utilized a standard Unity C# code review checklist focusing on:

- **Functionality:** the project needs to be cloneable and bootable after any change lands, and behavior between machines ought to be effectively identical.
- **Integrity:** if a new game object is added with references (most Unity objects have references), then all references must be intact after the inspector clones for review
- **User Experience:** the menus and interaction points of our menus and also of the main game itself should be legible to users and work the same way consistently.
- **Readability:** changes that modify or introduce C# scripts ought to adhere to standard C# programming conventions and code should be legible to the reviewer.

14 Inspection Results

Date	Item Inspected	Flaws Discovered	Resolution
Feb. 25, 2026	Wizard object, wand object, and path drawing system	The wizard and wand assets appeared correctly. The path	Missing .meta files were committed shortly after. The .gitignore was also

Date	Item Inspected	Flaws Discovered	Resolution
		drawing mechanic worked from the wand tip, but Unity .meta files were missing or inconsistent, causing some references to break after cloning.	adjusted so Unity metadata would be tracked correctly. Re-inspection confirmed references loaded properly.
Feb. 26, 2026	Initial projectile and spell creator system	The projectile prefab and projectile script were inspected. The projectile could be instantiated and follow drawn paths, but the early implementation was tightly coupled to Magic Missile behavior.	The system was later reworked into a generic Projectile class with spell-specific behavior objects. This improved extensibility for future spells.
Mar. 12, 2026	Wizard movement	Horizontal movement was inspected in the Unity editor. Basic movement worked, but the implementation was initially local-only and not suitable for multiplayer.	The movement system was accepted for the single-player prototype and later re-inspected after being migrated to Unity Netcode.
Mar. 24-25, 2026	Mana, health, health bar, and mana bar systems	Reviewers verified that health and mana values could be stored, updated, and displayed through UI bars. The main issue found was that later combat behavior needed to consistently call the health and mana APIs instead of duplicating state.	Spell casting and projectile damage were updated to use these systems. Later networking work converted health into a replicated network value.

Date	Item Inspected	Flaws Discovered	Resolution
Mar. 29-31, 2026	Reworked spell architecture	The group inspected the abstract Spell, SpellContext, ISpellBehavior, generic Projectile, and SpellCreator structure. The main discovery was that this structure made adding new spells much easier than the original Magic Missile-specific projectile code.	The reworked architecture was accepted. Future spells such as Fireball, Gust of Wind, Enlarge, Fog, Levitate, and Dimensional Door were implemented using or extending this structure.
Mar. 30-Apr. 1, 2026	Fireball and Gust of Wind spells	Fireball was inspected for projectile behavior, explosion behavior, and projectile destruction radius. Gust of Wind was inspected for directional placement and projectile redirection. Minor issues were found with spell interactions and projectile collision handling.	Gust of Wind and projectile behavior were corrected in follow-up commits. Re-inspection confirmed that wind could redirect projectiles and Fireball could destroy projectiles in its explosion radius.
Apr. 2, 2026	Enlarge and Cloud of Fog spells	Enlarge was inspected as a placement zone that modifies projectile scale. Cloud of Fog was inspected for cloud spawning, duration, and interaction with Gust of Wind. Reviewers confirmed the spell	Accepted after confirming that fog clouds could be displaced by wind and that both spell zones cleaned themselves up after their duration.

Date	Item Inspected	Flaws Discovered	Resolution
		effects were visible and temporary.	
Apr. 2, 2026	Levitate spell	Reviewers inspected behavior for lifting wizards and projectiles. The main concern was whether projectile movement resumed correctly after levitation ended.	The implementation pauses projectiles while lifted and then gives them a downward path when the effect ends. Re-inspection showed the spell behaved consistently enough for the prototype.
Apr. 2, 2026	Speed spell and Dimensional Door spell	Speed was inspected as a zone that modifies projectile speed. Dimensional Door was inspected for two-portal linking, teleporting projectiles/wizards, and redirecting projectiles out of the exit portal. Reviewers noted that portal placement and projectile direction needed to be intuitive.	Portal behavior was accepted after review. Dimensional Door sprites were added in a follow-up commit so the spell was visually identifiable.
Apr. 3-Apr. 10, 2026	Stone Wall and Shield spells	Stone Wall was inspected for placement preview, lifetime behavior, and collision with projectiles. Shield was inspected for prefab setup and initial behavior. Stone Wall was more complete; Shield appeared to have initial implementation but less complete	Stone Wall was accepted after projectile collision behavior was verified. Shield remained a lower-confidence feature and was treated as partially implemented compared with the other spell systems.

Date	Item Inspected	Flaws Discovered	Resolution
		integration than the other spells.	
Apr. 15, 2026	Main menu spellbook and spell loadout	Reviewers inspected MainMenuController, SpellbookSelection, SpellButtonUI, and SpellType. The menu allowed selecting up to four spells and saving them through PlayerPrefs. A spellbook error was found after the first implementation.	The spellbook error was fixed in a follow-up commit on Apr. 15. Re-inspection confirmed that the spellbook UI could open, close, and save selections.
Apr. 15-Apr. 25, 2026	Initial networking, Relay host/client flow, and network spawning	Reviewers inspected Unity Netcode setup, NetworkManagerUI, Relay join codes, host/client buttons, lobby status, wizard prefab registration, and spawn behavior. The main issues were ownership, local-only setup, and making sure each client controlled only their own wizard.	LocalPlayerSetup, LocalWizardSetup, and NetworkSpawner were added. Re-inspection confirmed that hosting, joining, spawning, and local ownership behavior worked more reliably.
Apr. 27, 2026	Final networking, health replication, and game-over screen	Reviewers inspected final networked movement, health, health bars, mana bars, projectile damage, and GameOverUI. The main flaw found was that game-over behavior needed to be shown correctly to	Health was updated to use server-authoritative damage and a client RPC for game-over display. Re-inspection confirmed that when a player reached zero health, both clients received the appropriate result screen.

Date	Item Inspected	Flaws Discovered	Resolution
		both players, with different win/loss text depending on which client died.	
Apr. 27, 2026	Final build	The final build and networking work were inspected together. Reviewers verified that the project could be built and that core gameplay was available in the build. Some Unity-generated files, logs, and build artifacts created large diffs.	The build was pushed for final delivery. The group accepted the large Unity diff as expected for a Unity project, while focusing inspection on scripts, scenes, prefabs, and gameplay behavior.

Table 1 - Inspection Results Table

V Recommendations and Conclusions

Most of our core components successfully passed review. Throughout the inspection process we identified several initial flaws, including missing Unity “.meta” files, tightly coupled projectile scripts, and local-only wizard movement. We were able to fix these issues, resulting in a generic and extensible projectile class and a fully migrated Unity Netcode movement system. The final build effectively delivers the primary requirements, notably the draw-to-cast pathing mechanic, synchronized health and energy systems, and a stable relay-based multiplayer architecture.

However, we did not finish all of our planned features. The Stone Wall spell, for example, was planned but is not fully functional in the final release due to time constraints and due to our prioritization of multiplayer networking features. Additionally, the inspection process revealed that the Shield spell needs more work, as the initial implementation is complete but it is less completely integrated compared to other core spells.

Based on these results, the recommended next action is finishing the integration of the Stone Wall and Shield spells so they function correctly within the networked multiplayer environment. Once these defensive features are fully implemented, they will need to be inspected according to the inspection procedures we’ve outlined above. Overall, our project is a successful, playable prototype that fulfills its core design goals, and we had a

lot of fun making it together and learned a lot along the way.

VI Project Issues

15 Open Issues

Divergence from Original Physics Design

The original project design document emphasized a heavy focus on the physics aspect of the game. We do use physics for some things like collision detection and gravity but for projectile movement we used a different implementation because it allowed us to prioritize adding more content and it also allowed us to implement the core functionality in a shorter time frame. The open issue is whether this drift will cause the product to drift from the design team's original vision over time, or if the game will be largely compatible with the design they made.

Repository Management and Build Artifacts

Unity generates a lot of files, logs, and intermediate build artifacts which can inflate the diffs on GitHub and sometimes add friction to the inspection process. We were able to fix the critical problems regarding the missing ".meta" files but the continuous adding/changing/removing of these intermediate Unity files remains an ongoing repository management issue that could be challenging for future maintainers of the project.

16 Waiting Room

Complete Integration of Defensive Spells

The Stone Wall and Shield spells have initial implementations but were not fully integrated due to us having prioritized the multiplayer networking features. Finishing the implementations for these spells so they function correctly in a networked environment is a high-priority feature for the next update.

Enhanced Physics Mechanics

The original design document put a heavy emphasis on the physics aspect of the game, but our implementation focused more on content and playability, with physics only being used for things like gravity and collisions. Future versions of the game could explore deeper physics interactions like the design team originally envisioned.

Expanded Offline Functionality

The current offline mode allows a player to cast basic spells if they lose connection mid-match to prevent crashes. Developing a dedicated single-layer practice arena or a game mode that lets you practice against AI could allow players to use our product in environments with poor connectivity or enable them to experiment with different loadouts without accruing losses on multiplayer mode.

Advanced Spell Combinations

The game currently supports some fun interactions between spells, like Gust of Wind pushing the Cloud of Fog and the Fireball spell destroying incoming projectiles.

Expanding this system to allow for more complex interactions and emergent behaviors could add more strategic depth to the deck-building and live combat systems.

Competitive Matchmaking

The current multiplayer architecture handles hosting and joining games and a Quick Match queue. Because the game is intended for both fun and competition, a future release of the game could introduce a ranked matchmaking system with skill-based ratings and leaderboards.

17 Ideas for Solutions

Version Control and Build Artifact Management

During our inspections we noted that Unity-generated files and build artifacts created unnecessarily large diffs. To resolve this, future maintainers could expand on our existing .gitignore template to reduce friction during feature inspections.

Networking the Defensive Spells

The Stone Wall and Shield spells are currently incomplete for networked multiplayer gameplay. Future maintainers could integrate these spells with Unity Netcode to ensure the placement spells are instantiated as server-authoritative NetworkObjects so that both players are able to cast, see, and interact with these important spell types.

Automated Testing

To satisfy requirement MS-3 (Automated Testing) the team could utilize the Unity Test Framework, which is commonly used for automated testing in Unity projects. It would help verify the core logic of the SpellCreator, mana consumption calculations, and base damage formulas independently of the network layer or physics engine.

Add More Physics

To reconcile our current implementation of the projectile spells with the design team's original physics-focused vision, the team could try writing new spell classes that use physics-based movement calculations. This would benefit a lot from automated testing to ensure there are no behavioral regressions between the old version and the new version.

18 Project Retrospective

What Worked Well

- **Modular Architecture:** building the spell system to use an abstract Spell architecture, including SpellContext and ISpellBehavior, and a generic Projectile class was very effective. It gave us a solid base on top of which we could design a wide variety of different spells.
- **Multiplayer Migration:** Migrating the wizard movement from our initial singleplayer local-only setup to a fully functional, network-authoritative movement system using Unity Netcode worked really well and was impressive for us all to see

in the inspection phase.

- **Core Gameplay Loop:** We ultimately delivered on the primary requirements, including the draw-to-cast pathing mechanic, the synchronized health and energy systems, and the multitude of spells available for players to use in combat.

Areas for Improvement

- **Version Control:** Early in development when we missed the “.meta” files it caused a lot of our entity references to break when cloning the repository and trying to run the game during the inspection phase. The diffs are also large due to excessive Unity file churn.

VII Glossary

A

- **Action Speed:** Derived stat calculated from attack speed affecting gameplay speed.

C

- **Casting:** The act of selecting and using a spell during a duel match.
- **Cloud of Fog:** A placement spell that creates fog and can be moved by wind.
- **Collision Detection:** System that determines when objects (projectiles, players) interact.

D

- **Dimensional Door:** A portal-based spell that teleports players or projectiles.
- **Damage Points:** The amount of health reduced when a player is hit.

E

- **Energy Points (Mana):** Resource required to cast spells; regenerates over time.

F

- **Fireball:** A projectile spell that explodes on impact and destroys nearby objects.

G

- **Gust of Wind:** A directional spell that redirects projectiles and affects fog.

H

- **Health Points (HP):** Determines how much damage a player can take before losing.

L

- **Lobby:** A staging area where players host or join matches before gameplay.

M

- **Magic Missile:** The main projectile spell following a user-drawn path.
- **Match:** A single duel between two players ending when one reaches 0 HP.

N

- **NetworkObject:** Unity Netcode object synchronized across players.

P

- **Projectile:** A moving object (spell) that follows a path and interacts with the environment.

R

- **Relay:** Unity service enabling multiplayer connections between players.

S

- **Shield:** Defensive spell with partial implementation.
- **Spell:** A magical ability that affects gameplay (damage or interaction).
- **Spell Deck:** UI system allowing players to select spells.
- **Spellbook:** Menu where players choose their spell loadout.
- **Stone Wall:** Defensive placement spell intended to block projectiles.

U

- **Unity Netcode:** Networking framework used for multiplayer implementation.

W

- **Wizard:** The player-controlled character in the duel.

VIII References / Bibliography

- [1] Robertson, S., and Robertson, J., **Mastering the Requirements Process**, Addison-Wesley, 2012.
- [2] Fowler, M., *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd ed., Pearson Education, 2004.

- [3] Unity Technologies, *Unity Manual*, Available: <https://docs.unity3d.com/>
- [4] Unity Technologies, *Netcode for GameObjects Documentation*, Available: <https://docs.unity3d.com/Packages/com.unity.netcode.gameobjects>
- [5] Group , *Magic Missile Wizard Duel Design Project Description Report*, CS 440, University of Illinois Chicago, Spring 2024.

IX Index

A

Action Speed 6, 34

C

Casting 5, 34

Cloud of Fog 34, 38, 42

Collision Detection 12, 16, 27

D

Dimensional Door 12, 34, 39

Damage Points 5

E

Energy Points 5, 6, 17

F

Fireball 34, 38, 42

G

Gust of Wind 10, 34, 38, 42

H

Health Points 5, 17, 29

L

Lobby 6, 16, 27, 35

M

Magic Missile 5, 7, 10, 34

Match 6

N

NetworkObject 12, 35

P

Projectile 7, 12, 15–16, 34

R

Relay 35

S

Shield 39, 41–43

Spell 5, 34

Spell Deck 8, 35

Spellbook 10, 21, 35

Stone Wall 10, 34, 41–43

T

Testing 12–33

U

Unity Netcode 35

W

Wizard 7, 34